

1. AMD GPU

AMD는 2019년 무렵부터 GPU 아키텍처 라인을 분리했다.

- **CDNA(Compute DNA)** : 데이터센터·AI·고성능컴퓨팅(HPC) 전용
- **RDNA(Radeon DNA)** : 게이밍 전용

1.1. CDNA (데이터센터 / AI)

특징

- Matrix Core(행렬 연산 전용 유닛) 강화
- FP64(64비트 정밀도 부동소수점) 연산 강화
- 그래픽 렌더링 파이프라인 없음
- AI 학습/추론과 과학 시뮬레이션에 집중

1.2. RDNA (게이밍)

특징

- 레이 트레이싱(빛의 경로를 추적해 사실적인 그림자·반사를 그리는 기술) 전용 하드웨어
- 그래픽 렌더링 파이프라인 탑재
- 디스플레이 출력 담당

2. CDNA (Instinct MI300x 시리즈) 아키텍처 핵심 구조

- MI300X는 하나의 칩이 아니라 여러 개의 작은 칩(chiplet, "칩렛" = 하나의 큰 칩을 여러 조각으로 나눠 따로 만든 뒤 이어붙인 것)을 이어붙인 구조
- 큰 칩 하나를 통째로 만들면 불량률이 높아지고 비용이 커지기 때문에, 작은 조각으로 나눠 만들고 필요한 만큼만 이어붙이는 것이 더 효율적이기 때문

MI300X (CDNA 3, 최대 구성)

└─ XCD × 8개 (Accelerator Complex Die, "연산 전담 조각")

└─ └─ 물리적으로 CU 40개 내장 → 수율 관리를 위해 38개만 활성화

└─ (8개 XCD × 38개 활성 CU = 총 304개 CU)

└─ IOD × 4개 (I/O Die, "입출력 전담 조각")

└─ AMD Infinity Fabric (조각들을 서로 연결하는 고속 통로, Part 6에서 자세히)

└─ HBM3 메모리 스택 × 8개 (192GB, 5.3TB/s)

MI300X 핵심 스펙	수치
Compute Unit(CU) 총 개수	304개
클럭	최대 2,100MHz
메모리	192GB HBM3, 대역폭 5.3TB/s
FP64 Matrix 연산 성능	163.4 TFLOP/s (MI250X 대비 1.7배)
FP8 연산 성능	약 2.6 PFLOP/s (2,614.9 TFLOP/s)

3. ROCm & HIP 생태계

NVIDIA GPU를 위한 CUDA가 있다면 AMD는 HIP이 존재한다.

HIP(Heterogeneous-computing Interface for Portability)

- "여러 하드웨어를 오가며 쓸 수 있게 만든 인터페이스"은 AMD GPU용 C++ 런타임 API이자 커널 (GPU에서 실행되는 프로그램 조각) 작성 언어다.
- NVIDIA CUDA 코드를 AMD GPU로 옮기기 쉽게 만드는 것이 명확한 설계 목적이다.

구분	NVIDIA CUDA	AMD ROCm/HIP
API 구조	드라이버 API와 런타임 API가 분리	하나로 통합된 단일 API
수학/내장 함수	기준	이름 규칙만 다르고 함수 시그니처는 거의 동일 → 대부분 코드는 변환 작업이 사실상 불필요
변환 도구	해당 없음	HIPIFY(오픈소스, ROCm의 일부) - hipify-clang(Clang 기반 AST 파서, CUDA 설치 필요) - hipify-perl(패턴 매칭, CUDA 설치 불필요) 2종
변환 범위	-	CUDA 런타임 코드 대부분을 자동 변환 가능. 다만 아키텍처 기능 조희나 HIP이 지원하지 않는 일부 CUDA 기능은 수작업 필요
완전 자동 치환 여부	-	AMD가 공식적으로 "드롭인 대체재(drop-in replacement)가 아니다"라고 명시 포팅을 도와주는 도구이지 100% 자동화 도구는 아니다.

4. 하드웨어 아키텍처

- MI300X는 8개의 "연산 전담 조각(XCD)"이 4개의 "입출력 전담 조각(IOD)" 위에 쌓여 Infinity Fabric 으로 이어붙은 칩렛 GPU다.
- XCD 하나에 물리적으로 CU가 40개씩 있지만 수율 관리 때문에 38개만 켜져 있어,
총 물리 **CU 320개** 중 활성 **CU 304개**(Matrix Core 1,216개, 스트림 프로세서 19,456개)를 갖는다.

- GPU에서 실제로 "한 덩어리로 실행되는 진짜 단위"는 스레드(work-item) 개별이 아니라 **64개 스레드를 묶은 Wavefront**다.

4.1. CDNA 칩렛 구조 (XCD & IOD)

- **XCD(Accelerator Complex Die, "가속 연산 전담 조각")**
 - CU들을 모아놓은 연산 전용 칩렛
- **IOD(I/O Die, "입출력 전담 조각")**
 - Infinity Cache와 HBM3 메모리 컨트롤러를 담고 있는 입출력 전용 칩렛

MI300X는 이 두 종류의 조각을 3차원으로 쌓아 하나의 패키지로 만든다.

MI300X 칩렛 구성

- |— XCD × 8개 (TSMC 5nm 공정)
 - | — XCD당 물리 CU 40개 → 38개만 활성화 (수율 관리)
 - 총 물리 CU 320개 / 활성 CU 304개
 - Matrix Core $304 \times 4 = 1,216$ 개
 - 스트림 프로세서 $304 \times 64 = 19,456$ 개
- |— IOD × 4개 (TSMC 6nm 공정, Infinity Cache + HBM3 컨트롤러 내장)
 - | — AMD Infinity Fabric (XCD·IOD를 서로 잇는 고속 통로)

4.2. GPU Partitioning Mode (하나의 GPU를 여러 개처럼 나눠 쓰기)

MI300X는 하나의 물리 GPU를 논리적으로 여러 개로 쪼개서 서로 다른 작업에 나눠줄 수 있다(NVIDIA의 MIG과 유사한 개념). 이를 파티셔닝 모드라 부른다.

모드	구성	GPU당 CU / 메모리
SPX (Single Partition X-celerator) - Default	1개로 통합	304 CU / 192GB
DPX (Dual Partition X-celerator)	2개로 분할	파티션당 152 CU / 96GB
CPX (Core Partitioned X-celerator)	8개로 분할	파티션당 38 CU / 24GB

4.2. CU (Compute Unit)

- CU(Compute Unit)
 - GPU 안에서 실제 계산을 수행하는 기본 하드웨어 블록.
 - 내부에는 SIMD(Single Instruction, Multiple Data, "한 개의 명령으로 여러 데이터를 동시에 처리") 유닛들과 행렬 곱셈(AI의 핵심 연산)을 전담하는 **Matrix Core**가 들어있다.
 - CU 여러 개가 하나의 XCD 안에서 **ACE(Asynchronous Compute Engine, "비동기 작업 배분기")**를 통해 작업(workgroup)을 배정받는다.

- XCD 하나당 ACE 4개가 존재하며, 4개의 ACE가 협력해 XCD 내 40개의 CU들에게 작업을 분배한다.
- 같은 XCD 안의 CU들은 **4MB L2 캐시**를 함께 공유한다.

4.3. 실행 단위 계층 트리

- CDNA에서 Wavefront는 **64개** 스레드 단위, RDNA(게이밍용)는 **32개** 단위다.
- 이 숫자만큼의 스레드가 SIMD 유닛에 한 번에 매핑되어. 같은 명령을 동시에 실행한다.
- Workgroup(=NVIDIA의 Thread Block에 대응)은 항상 하나의 CU 안에서만 실행되며, 그래서 같은 Workgroup에 속한 스레드들끼리는 CU 내부의 LDS(Local Data Share)를 함께 쓸 수 있다.

```

Grid / NDRange ("이번 커널이 처리할 문제 전체")
├── Workgroup ("같은 CU에 지정되는 작업 묶음, LDS 공유")
│   ├── Wavefront ("64개 스레드가 한 몸처럼 SIMD에서 실행되는 진짜 실행 단위")
│   │   └── Work-item("계산을 수행하는 최소 단위 하나")

```

4.3.1. AMD(HIP) vs NVIDIA(CUDA) 실행 단위 용어 비교표

계층	AMD / HIP 용어	NVIDIA / CUDA 용어	묶음 크기	실행되는 위치
최소 단위	Work-item	Thread	1개	연산 유닛(SIMD 레인) 1개
진짜 실행 단위(lockstep)	Wavefront	Warp	- CDNA(MI300X): 64개 - RDNA: 32개 - NVIDIA: 항상 32개	- CU(AMD) - SM(NVIDIA) 내부의 SIMD 유닛
중간 묶음	Workgroup	Thread Block	최대 16 Wavefront = 1,024 work-item	단일 CU(AMD) 단일 SM(NVIDIA) 에 통째로 배정, LDS/Shared Memory 공유
중간 묶음의 묶음	-	Thread Block Cluster (Hopper 이상)	-	여러 SM에 걸친 Thread Block들을 하나로 묶어 서로 협력(Distributed Shared Memory 등)
최상위 문제 공간	Grid / NDRange	Grid	커널 전체	커널 전체

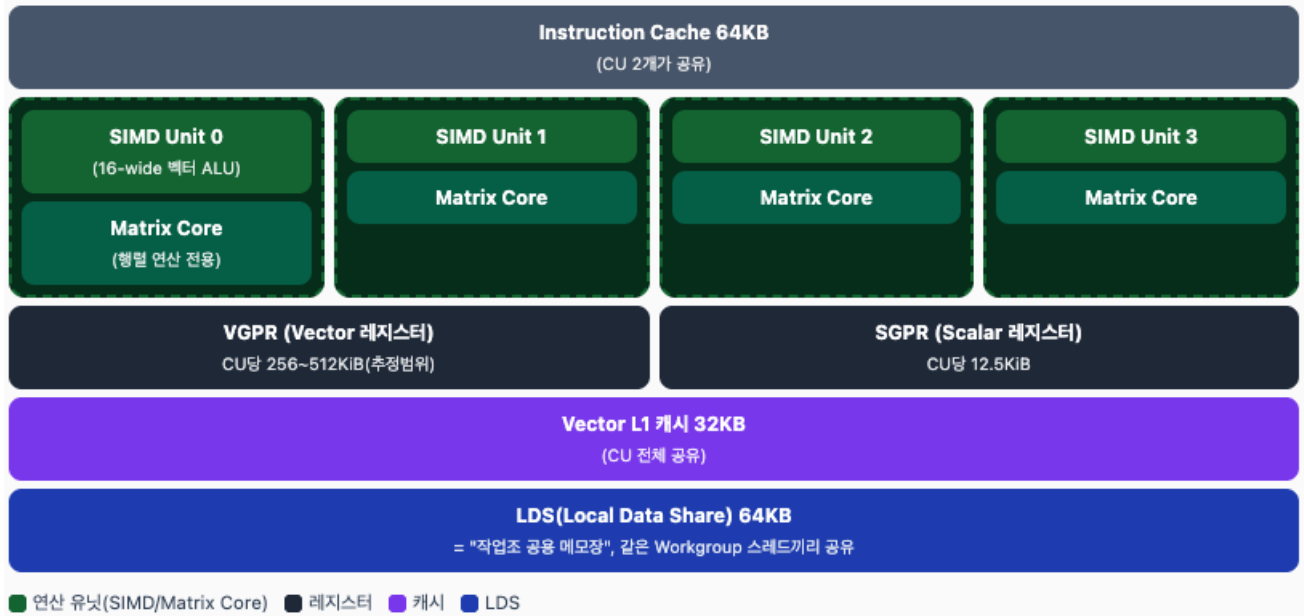
4.4. AMD GPU 메모리 계층 구조

계층	스코프(누가 공유 하나)	위치	속도감	크기 (MI300X/CDNA3 기준)
VGPR/SGPR (레지스터)	스레드/Wavefront 전용	On-chip (CU 내부)	최고속(1사이클급)	CU당 SGPR 12.5KiB, VGPR 256~512KiB
LDS (Local Data Share)	같은 Workgroup(=같은 CU)	On-chip (CU 내부)	레지스터 다음으로 빠름, HBM 대비 한 자릿수 이상 빠름	CU당 64KB
Vector L1 캐시	CU 전용	On-chip (CU 내부)	빠름	CU당 32KB
Instruction 캐시	CU 2개가 공유	On-chip	빠름	64KB, 8-way set-associative
L2 캐시	같은 XCD 내 CU 38개 전체가 공유	On-chip (XCD 내부)	중간	XCD당 4MB, 16-way set-associative
Infinity Cache (LLC)	GPU 전체(모든 XCD)가 공유	On-chip이지만 IOD(I/O Die)에 위치	L2보다 느리지만 HBM보다 훨씬 빠름	256MB, 16-way set-associative, XCD로의 집계 대역폭 17.2TB/s
Global Memory (HBM3)	GPU 전체	Off-chip (별도의 메모리 스택)	가장 느림(그래도 일반 DDR 보다는 훨씬 빠름)	192GB, 5.3TB/s

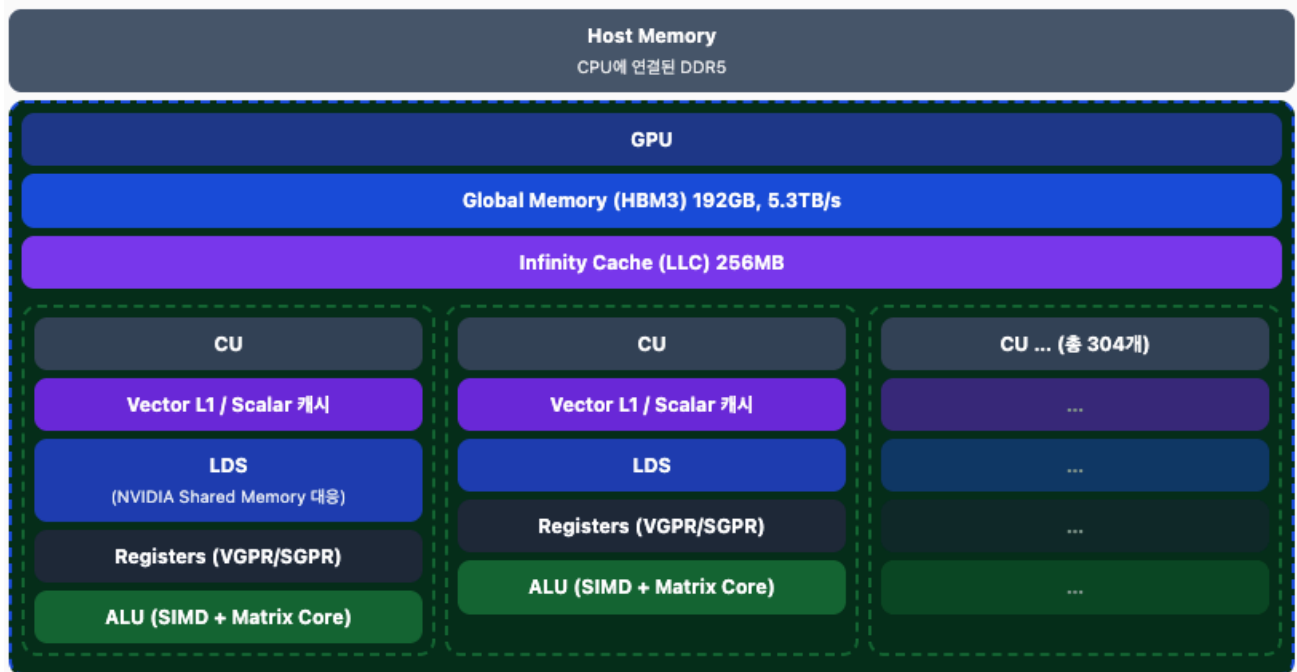
- MI300X 패키지 전체의 물리적 데이터 경로 (NVIDIA GPU DRAM→L2→GPC 도식에 대응)



- CU(Compute Unit) 내부 구조 (NVIDIA SM 내부 도식에 대응)



- 스코프별 메모리 배치: Host ↔ GPU ↔ CU (NVIDIA host/global/SM 도식에 대응)



4.4.1 레지스터(VGPR/SGPR)와 Occupancy

- VGPR(Vector General Purpose Register)**
 - Wavefront 안의 64개 스레드가 "각자" 갖는 벡터 계산용 레지스터
 - 크기 : 일반적으로 4개의 64 또는 128KiB => CU당 256~512KiB
- SGPR(Scalar General Purpose Register)**
 - Wavefront 전체가 "공통으로" 쓰는 스칼라(단일 값) 레지스터
 - 크기 : 12.5KiB

4.4.2 LDS (Local Data Share)

- NVIDIA의 Shared Memory에 대응하는 개념이다.
- 같은 Workgroup(=같은 CU에서 실행)에 속한 스레드들끼리 데이터를 빠르게 주고받을 수 있는 온칩 (on-chip, 칩 내부에 있어서 매우 빠른) 메모리

4.4.3. Cache

MI300X(CDNA3) 기준

Vector L1 캐시

- CU 하나 당 32KB 를 가진다.

Instruction 캐시

- 다음에 실행할 명령어를 미리 담아두는 캐시
- CU 2개가 64KB를 공유

L2 캐시

- CU 38개(XCD 하나 안의 활성 CU 전체)가 **4MB**를 공유

Infinity Cache

- CU나 XCD가 아니라 **IOD(I/O Die)**에 위치한, GPU 전체(8개 XCD 전부)가 함께 쓰는 **256MB** 크기의 캐시
- 대역폭 : 최대 17.2TB/s (XCD 들과 연결된다.)

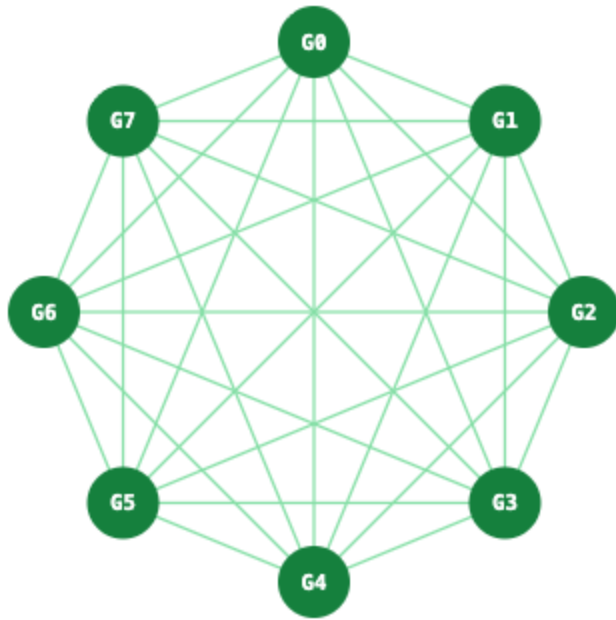
5. GPU 간 연결 - Interconnect

- MI300X 8-GPU 서버는 스위치 칩 없이 GPU 8개가 서로 직접 연결되는 완전 메시(fully-meshed, all-to-all direct-connect) 구조다.
- GPU 1개당 7개의 Infinity Fabric 링크(각 128GB/s)로 나머지 7개 GPU 전부와 1:1 직결된다.
- PCIe Gen5 x16(128GB/s) 링크 1개가 호스트 CPU로 연결된다.

5.1. Infinity Fabric / xGMI — GPU 전용 고속 통로

- **Infinity Fabric** (xGMI라고도 부름)은 AMD GPU끼리 데이터를 직접 주고받는 전용 고속 통로다.
- GPU 1개당 7개의 **Infinity Fabric** 링크를 가지며, 각 링크는 **128GB/s** 양방향 대역폭을 낸다.
 - 7개 링크로 나머지 7개 GPU 전부와 1:1로 직접 연결되어, GPU 1개 기준 총 **896GB/s** ($7 \times 128\text{GB/s}$)의 GPU-GPU 대역폭을 갖는다.

5.2. GPU Connection Topology (Switchless All-to-All Mesh)



5.3. NVLink/NVSwitch 대비 AMD 방식의 아키텍처적 차이

구분	NVIDIA (NVLink + NVSwitch)	AMD (Infinity Fabric, MI300X 기준)
연결 방식	별도의 중앙 스위치 칩(NVSwitch)을 거쳐 GPU들이 연결되는 스위치 기반 (switched) 구조	스위치 칩 없이 GPU끼리 직접 점대점으로 연결되는 완전 메시(switchless mesh) 구조
GPU당 연결 수	스위치로 집중(모든 GPU가 스위치와 연결)	나머지 GPU 개수만큼 직접 링크 (MI300X는 7개)
확장성(스케일업)	스위치를 늘리면 더 많은 GPU/노드를 하나의 대역폭 도메인으로 묶기 유리	GPU 개수가 늘어날수록 GPU당 필요한 직접 링크 수도 함께 늘어나는 구조 — 완전 메시 특유의 확장 한계 존재
비용/복잡도	별도 스위치 칩 비용과 설계 복잡도 추가	스위치 칩이 없어 그만큼 부품·비용이 단순해짐
대역폭 균일성	스위치를 통하더라도 설계상 균일한 대역폭 제공 목표	MI300X 세대부터는 모든 GPU 쌍이 동일하게 128GB/s(1홉)로 균일

- rocm-smi

```
===== ROCm System Management Interface =====
Concise Info
=====
```

Device	Node	IDs (DID, GUID)	Temp (Junction)	Power (Socket)	Partitions (Mem, Compute, ID)	SCLK	MCLK	Fan	Perf	PwrCap	VRAM%	GPU%
0	2	0x74a1, 33267	45.0°C	145.0W	NPS1, SPX, 0	132Mhz	900Mhz	0%	auto	750.0W	0%	0%
1	3	0x74a1, 23018	44.0°C	137.0W	NPS1, SPX, 0	132Mhz	900Mhz	0%	auto	750.0W	0%	0%
2	4	0x74a1, 29122	47.0°C	137.0W	NPS1, SPX, 0	132Mhz	900Mhz	0%	auto	750.0W	0%	0%
3	5	0x74a1, 43483	44.0°C	136.0W	NPS1, SPX, 0	132Mhz	900Mhz	0%	auto	750.0W	0%	0%
4	6	0x74a1, 8594	43.0°C	140.0W	NPS1, SPX, 0	132Mhz	900Mhz	0%	auto	750.0W	0%	0%
5	7	0x74a1, 63883	49.0°C	144.0W	NPS1, SPX, 0	132Mhz	900Mhz	0%	auto	750.0W	0%	0%
6	8	0x74a1, 53667	45.0°C	138.0W	NPS1, SPX, 0	131Mhz	900Mhz	0%	auto	750.0W	0%	0%
7	9	0x74a1, 2490	43.0°C	137.0W	NPS1, SPX, 0	132Mhz	900Mhz	0%	auto	750.0W	0%	0%

```
===== End of ROCm SMI Log =====
```


- rocm-smi --showtopo

```

===== ROCm System Management Interface =====
===== Weight between two GPUs =====
GPU0  GPU0    GPU1    GPU2    GPU3    GPU4    GPU5    GPU6    GPU7
GPU0  0        15      15      15      15      15      15      15
GPU1  15       0       15      15      15      15      15      15
GPU2  15      15      0       15      15      15      15      15
GPU3  15      15      15      0       15      15      15      15
GPU4  15      15      15      15      0       15      15      15
GPU5  15      15      15      15      15      0       15      15
GPU6  15      15      15      15      15      15      0       15
GPU7  15      15      15      15      15      15      15      0

===== Hops between two GPUs =====
GPU0  GPU0    GPU1    GPU2    GPU3    GPU4    GPU5    GPU6    GPU7
GPU0  0        1        1        1        1        1        1        1
GPU1  1        0        1        1        1        1        1        1
GPU2  1        1        0        1        1        1        1        1
GPU3  1        1        1        0        1        1        1        1
GPU4  1        1        1        1        0        1        1        1
GPU5  1        1        1        1        1        0        1        1
GPU6  1        1        1        1        1        1        0        1
GPU7  1        1        1        1        1        1        1        0

===== Link Type between two GPUs =====
GPU0  GPU0    GPU1    GPU2    GPU3    GPU4    GPU5    GPU6    GPU7
GPU0  0        XGMI    XGMI    XGMI    XGMI    XGMI    XGMI    XGMI
GPU1  XGMI      0       XGMI    XGMI    XGMI    XGMI    XGMI    XGMI
GPU2  XGMI    XGMI      0       XGMI    XGMI    XGMI    XGMI    XGMI
GPU3  XGMI    XGMI    XGMI      0       XGMI    XGMI    XGMI    XGMI
GPU4  XGMI    XGMI    XGMI    XGMI      0       XGMI    XGMI    XGMI
GPU5  XGMI    XGMI    XGMI    XGMI    XGMI      0       XGMI    XGMI
GPU6  XGMI    XGMI    XGMI    XGMI    XGMI    XGMI      0       XGMI
GPU7  XGMI    XGMI    XGMI    XGMI    XGMI    XGMI    XGMI      0

===== Numa Nodes =====
GPU[0] : (Topology) Numa Node: 0
GPU[0] : (Topology) Numa Affinity: 0
GPU[1] : (Topology) Numa Node: 0
GPU[1] : (Topology) Numa Affinity: 0
GPU[2] : (Topology) Numa Node: 0
GPU[2] : (Topology) Numa Affinity: 0
GPU[3] : (Topology) Numa Node: 0
GPU[3] : (Topology) Numa Affinity: 0
GPU[4] : (Topology) Numa Node: 1
GPU[4] : (Topology) Numa Affinity: 1
GPU[5] : (Topology) Numa Node: 1
GPU[5] : (Topology) Numa Affinity: 1
GPU[6] : (Topology) Numa Node: 1
GPU[6] : (Topology) Numa Affinity: 1
GPU[7] : (Topology) Numa Node: 1
GPU[7] : (Topology) Numa Affinity: 1
===== End of ROCm SMI Log =====

```